

Dialysis

From air pressure to digits: a DTMF decoder, phase by phase

September 2026

Team Noise

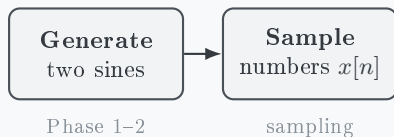
The whole idea in one picture



Generate
two sines

Phase 1-2

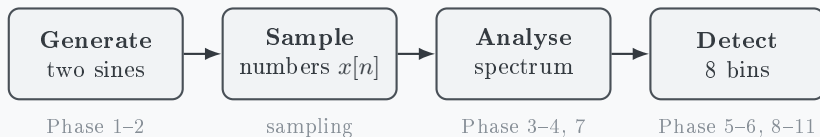
The whole idea in one picture



The whole idea in one picture



The whole idea in one picture



The whole idea in one picture

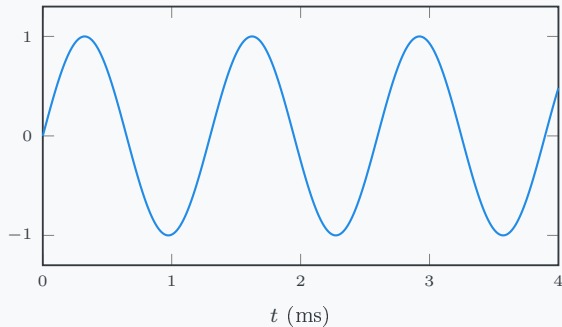


The whole idea in one picture



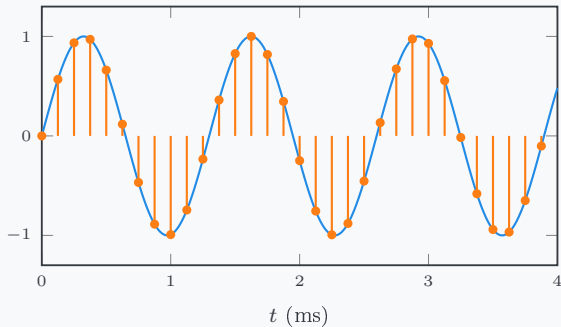
The same core loop serves the keypad, an imported `.wav`, and the live microphone.

Phase 1a – Sound becomes numbers



Continuous: a voltage that swings with the air pressure (here 770 Hz).

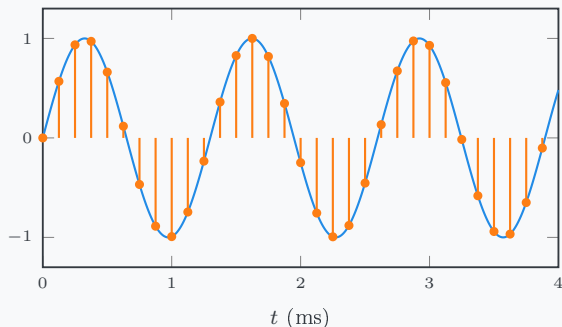
Phase 1a – Sound becomes numbers



Continuous: a voltage that swings with the air pressure (here 770 Hz).

Sampled: the ADC reads it every $1/f_s$ seconds, $x[n] = x(n/f_s)$. At $f_s = 8000$ Hz that is 8 numbers per ms.

Phase 1a – Sound becomes numbers



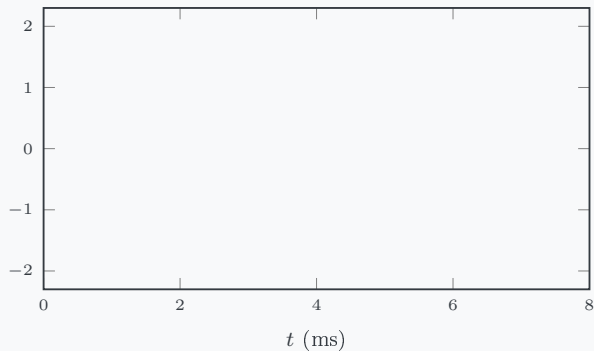
Continuous: a voltage that swings with the air pressure (here 770 Hz).

Sampled: the ADC reads it every $1/f_s$ seconds, $x[n] = x(n/f_s)$. At $f_s = 8000$ Hz that is 8 numbers per ms.

Nyquist: only $f < f_s/2$ survives. Highest DTMF tone is 1633 Hz < 4000 Hz ✓

Phase 1b – Synthesizing a DTMF tone

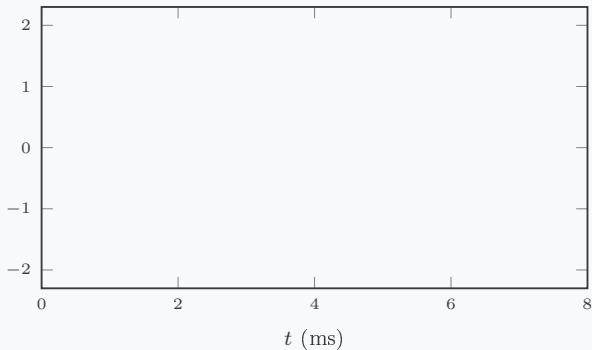
	1	2	3	A
697	4	5	6	B
770	7	8	9	C
852	*	0	#	D
941				



Phase 1b – Synthesizing a DTMF tone

	1209	1336	1477	1633
697	1	2	3	A
770	4	5	6	B
852	7	8	9	C
941	*	0	#	D

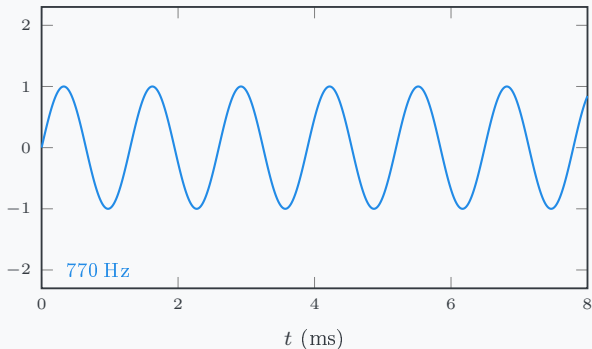
key “5” = row 770 Hz + column 1336 Hz



Phase 1b – Synthesizing a DTMF tone

	1209	1336	1477	1633
697	1	2	3	A
770	4	5	6	B
852	7	8	9	C
941	*	0	#	D

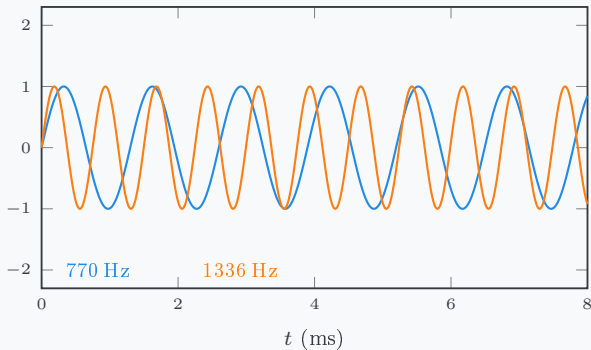
key “5” = row 770 Hz + column 1336 Hz



Phase 1b – Synthesizing a DTMF tone

	1209	1336	1477	1633
697	1	2	3	A
770	4	5	6	B
852	7	8	9	C
941	*	0	#	D

key “5” = row 770 Hz + column 1336 Hz

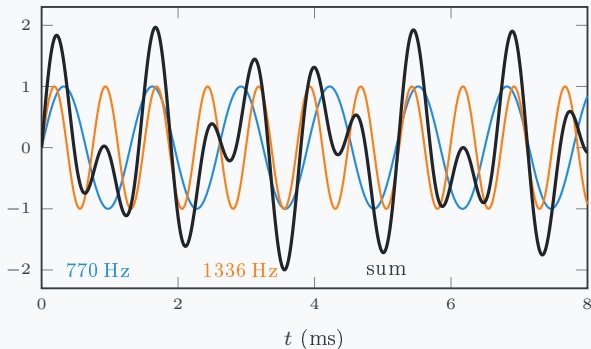


Phase 1b – Synthesizing a DTMF tone

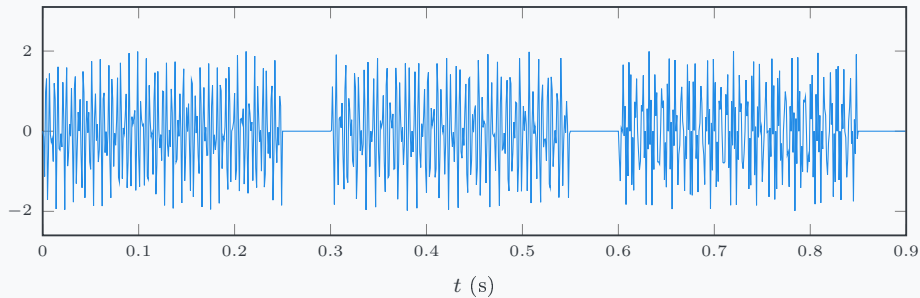
	1209	1336	1477	1633
697	1	2	3	A
770	4	5	6	B
852	7	8	9	C
941	*	0	#	D

key “5” = row 770 Hz + column 1336 Hz

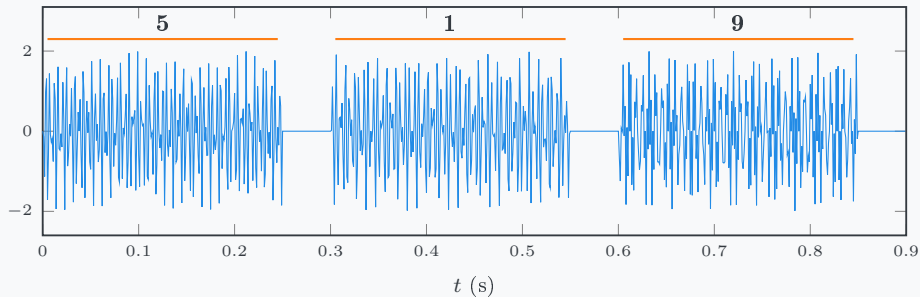
$$x[n] = \sin \frac{2\pi 770n}{f_s} + \sin \frac{2\pi 1336n}{f_s}$$



Phase 2 – Digits become a waveform

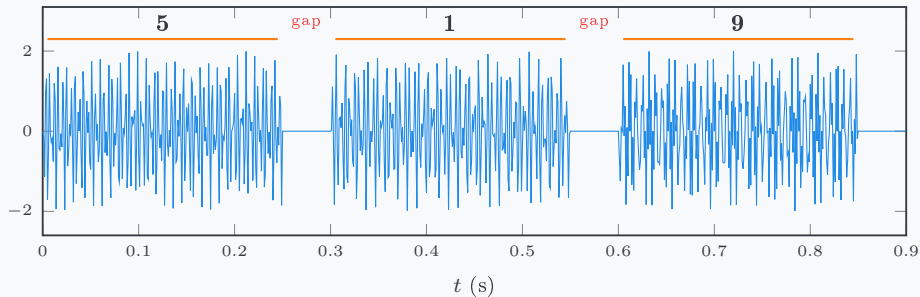


Phase 2 – Digits become a waveform



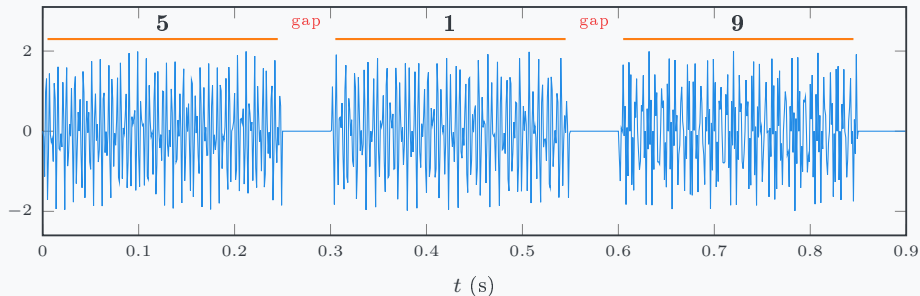
`sequence_to_wav: tone (250 ms) →`

Phase 2 – Digits become a waveform



`sequence_to_wav`: tone (250 ms) $\rightarrow$ silence (50 ms, exact zeros) $\rightarrow$ tone $\rightarrow \dots$

Phase 2 – Digits become a waveform



`sequence_to_wav`: tone (250 ms) $\rightarrow$ silence (50 ms, exact zeros) $\rightarrow$ tone $\rightarrow \dots$ then
play_array (DAC $\rightarrow$ speaker) or save_wav (16-bit PCM).

Phase 3 – Which frequencies are inside?

1. To test one frequency f : multiply $x[n]$ by a wave at f and add up.
Match $\Rightarrow$ big, mismatch $\Rightarrow$ cancels.

Phase 3 – Which frequencies are inside?

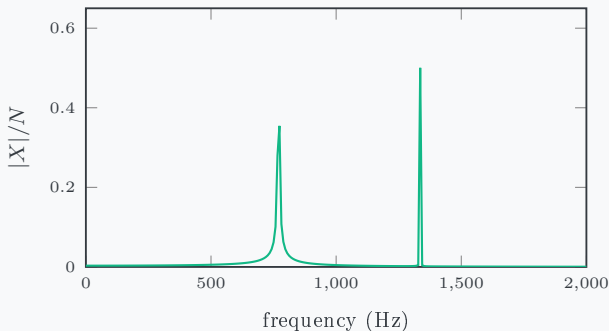
1. To test one frequency f : multiply $x[n]$ by a wave at f and add up.

Match $\Rightarrow$ big, mismatch $\Rightarrow$ cancels.

2. A lone sine is blind to phase, so correlate with $\cos$ *and* $\sin$ at once: the complex exponential.

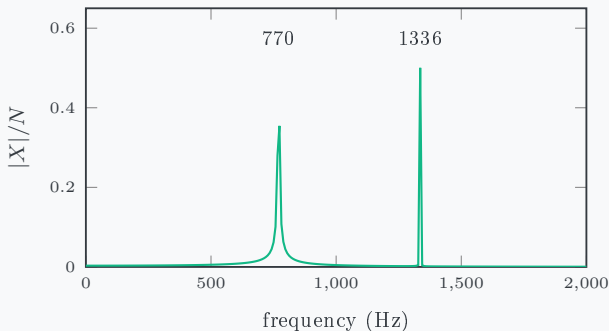
Phase 3 – Which frequencies are inside?

1. To test one frequency f : multiply $x[n]$ by a wave at f and add up. Match $\Rightarrow$ big, mismatch $\Rightarrow$ cancels.
2. A lone sine is blind to phase, so correlate with $\cos$ *and* $\sin$ at once: the complex exponential.
3. $X(k) = \sum_n x[n] e^{-j2\pi kn/N}$: one *bin* of the DFT. Do it for every k with the FFT ($N \log_2 N$, hand-written radix-2).



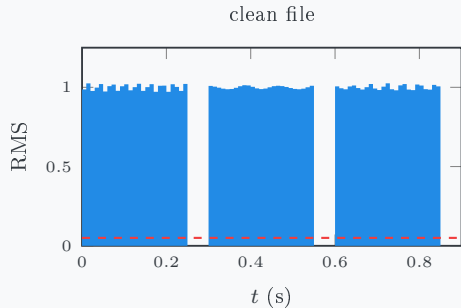
Phase 3 – Which frequencies are inside?

1. To test one frequency f : multiply $x[n]$ by a wave at f and add up.
Match $\Rightarrow$ big, mismatch $\Rightarrow$ cancels.
2. A lone sine is blind to phase, so correlate with $\cos$ *and* $\sin$ at once: the complex exponential.
3. $X(k) = \sum_n x[n] e^{-j2\pi kn/N}$: one *bin* of the DFT. Do it for every k with the FFT ($N \log_2 N$, hand-written radix-2).



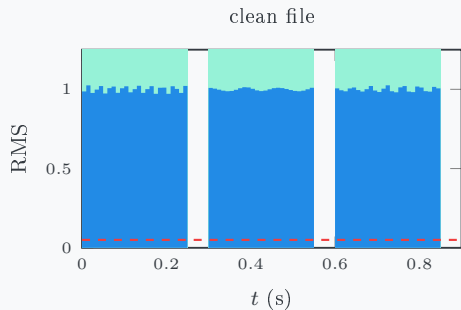
spectrum of key “5” from our own `compute_spectrum`

Phase 4 – Segmentation: where are the tones?



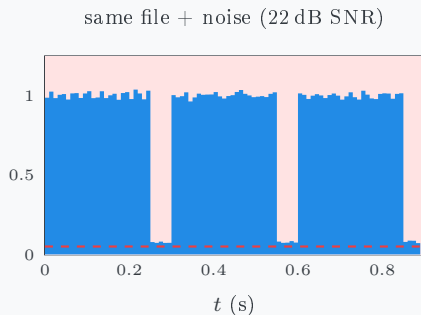
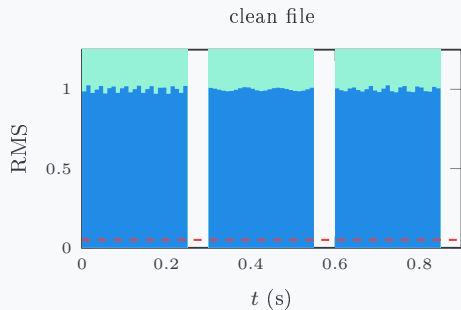
Rule: chunk is “loud” if $\text{RMS} > 5\%$ of the loudest 10 ms chunk (dashed).

Phase 4 – Segmentation: where are the tones?



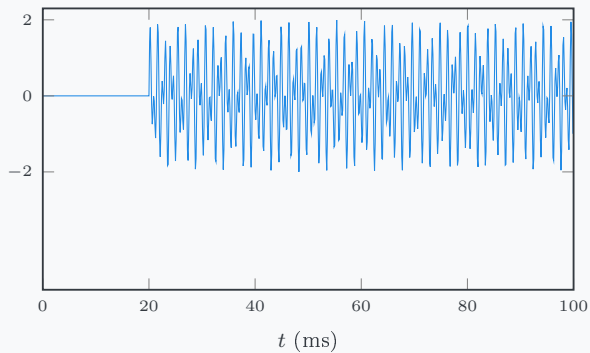
Rule: chunk is “loud” if $\text{RMS} > 5\%$ of the loudest 10 ms chunk (dashed). Clean: 3 regions.

Phase 4 – Segmentation: where are the tones?



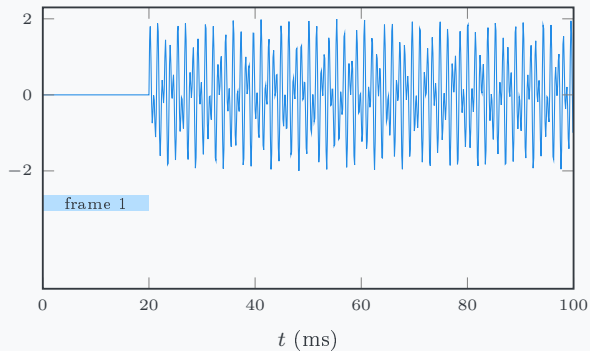
Rule: chunk is “loud” if $\text{RMS} > 5\%$ of the loudest 10 ms chunk (dashed). Clean: 3 regions.
Noisy: 1 giant region, decoding fails. A fixed threshold cannot work $\Rightarrow$ the live engine measures the noise floor.

Decoding engine (1) – Cutting the audio into frames



Silence, then key “5”.

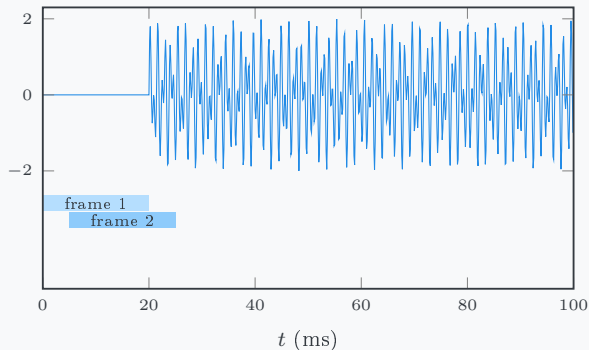
Decoding engine (1) – Cutting the audio into frames



Silence, then key “5”.

Frame = 20 ms
(160 samples at 8 kHz,
960 at 48 kHz).

Decoding engine (1) – Cutting the audio into frames

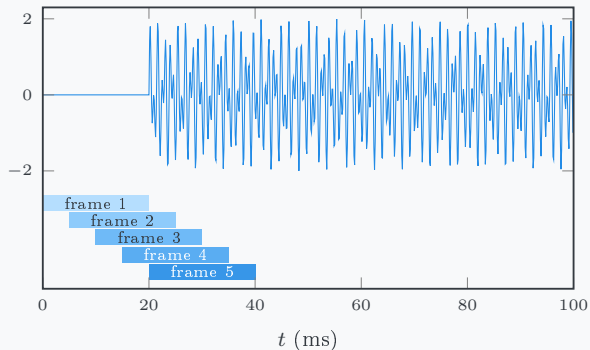


Silence, then key “5”.

Frame = 20 ms
(160 samples at 8 kHz,
960 at 48 kHz).

Hop = 5 ms: the next frame
starts 5 ms later, so frames
overlap 75%.

Decoding engine (1) – Cutting the audio into frames

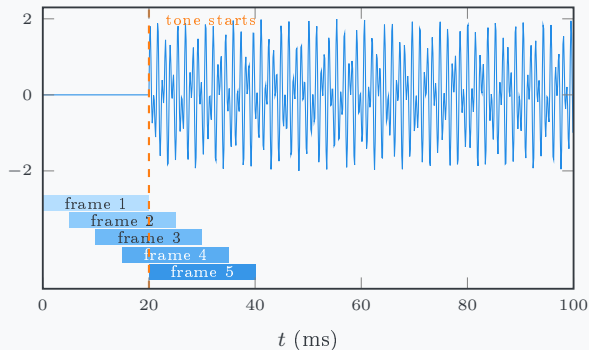


Silence, then key “5”.

Frame = 20 ms
(160 samples at 8 kHz,
960 at 48 kHz).

Hop = 5 ms: the next frame
starts 5 ms later, so frames
overlap 75%.

Decoding engine (1) – Cutting the audio into frames



Silence, then key “5”.

Frame = 20 ms
(160 samples at 8 kHz,
960 at 48 kHz).

Hop = 5 ms: the next frame
starts 5 ms later, so frames
overlap 75%.

Frame 5 is the first one fully
inside the tone.

200 frames per second $\Rightarrow$ 5 ms
timing.

Decoding engine (2) – How much 770 Hz is in a frame?

1. Take one frame of key “5”
(160 samples).

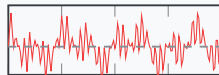
Decoding engine (2) – How much 770 Hz is in a frame?

1. Take one frame of key “5” (160 samples).
2. Multiply every sample by a 770 Hz wave. Match: products stay positive. 941 Hz: up and down, cancels.

$$x[n] \cdot \sin(2\pi 770 n / f_s)$$



$$x[n] \cdot \sin(2\pi 941 n / f_s)$$



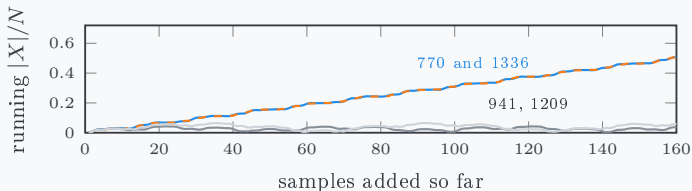
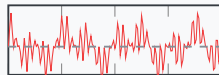
Decoding engine (2) – How much 770 Hz is in a frame?

1. Take one frame of key “5” (160 samples).
2. Multiply every sample by a 770 Hz wave. Match: products stay positive. 941 Hz: up and down, cancels.
3. Add them up (sine *and* cosine, so the tone’s phase doesn’t matter). The running total only grows for 770 and 1336.

$$x[n] \cdot \sin(2\pi 770 n / f_s)$$



$$x[n] \cdot \sin(2\pi 941 n / f_s)$$



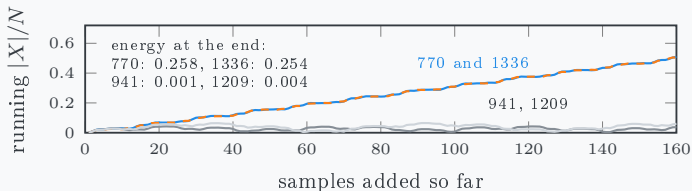
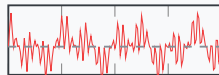
Decoding engine (2) – How much 770 Hz is in a frame?

1. Take one frame of key “5” (160 samples).
2. Multiply every sample by a 770 Hz wave. Match: products stay positive. 941 Hz: up and down, cancels.
3. Add them up (sine *and* cosine, so the tone’s phase doesn’t matter). The running total only grows for 770 and 1336.
4. Energy = $(|\text{total}|/N)^2$

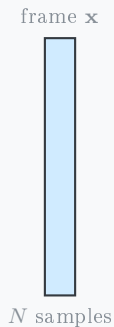
$$x[n] \cdot \sin(2\pi 770 n / f_s)$$



$$x[n] \cdot \sin(2\pi 941 n / f_s)$$

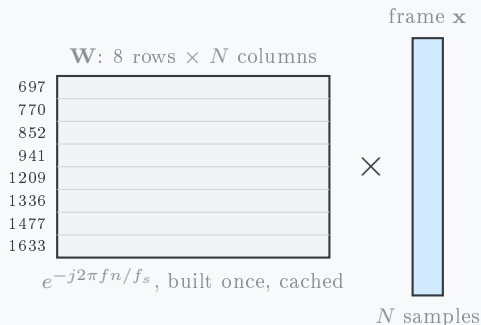


Decoding engine (3) – All 8 frequencies in one step



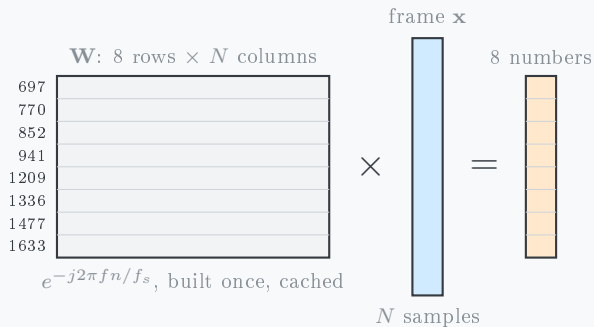
One frame $\mathbf{x}$.

Decoding engine (3) – All 8 frequencies in one step



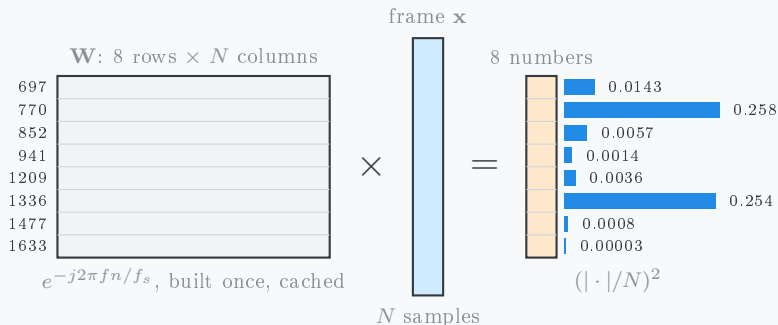
One frame $\mathbf{x}$. One row of $\mathbf{W}$ per DTMF frequency.

Decoding engine (3) – All 8 frequencies in one step



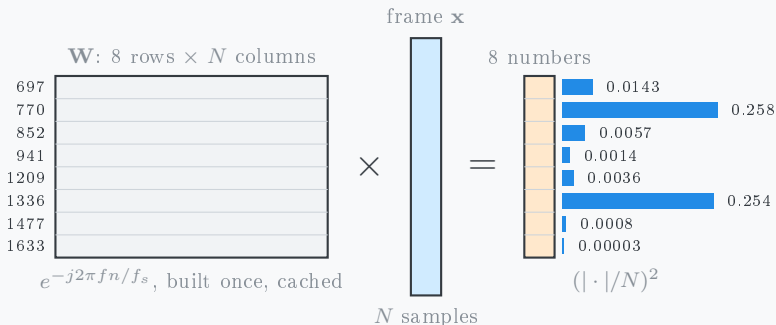
One frame $\mathbf{x}$. One row of $\mathbf{W}$ per DTMF frequency. One product gives all 8 correlations.

Decoding engine (3) – All 8 frequencies in one step



One frame $\mathbf{x}$. One row of $\mathbf{W}$ per DTMF frequency. One product gives all 8 correlations.
Squared size = energy per frequency.

Decoding engine (3) – All 8 frequencies in one step



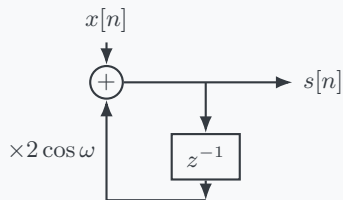
One frame $\mathbf{x}$. One row of $\mathbf{W}$ per DTMF frequency. One product gives all 8 correlations.
Squared size = energy per frequency.

frame_bin_powers: all frames at once, frames @ W.T.

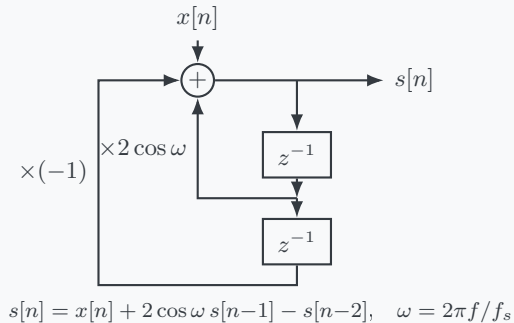
Phase 5 – Same numbers, filter form: Goertzel



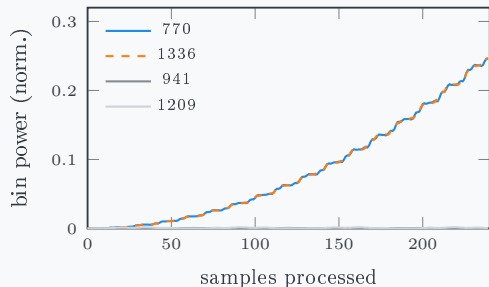
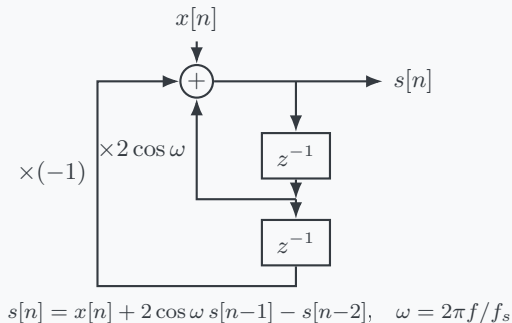
Phase 5 – Same numbers, filter form: Goertzel



Phase 5 – Same numbers, filter form: Goertzel

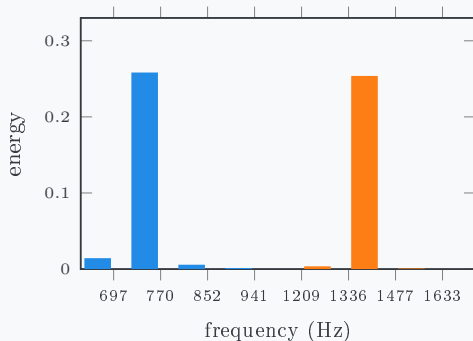


Phase 5 – Same numbers, filter form: Goertzel



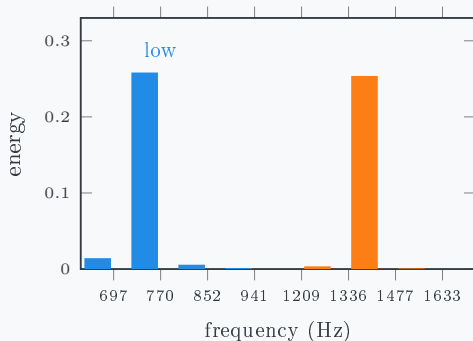
Key “5”: the two matching filters grow, the others stay flat. Cost: 1 multiply + 2 adds per sample.

Decoding with Goertzel – from 8 energies to a digit



Energies of one frame of “5”.

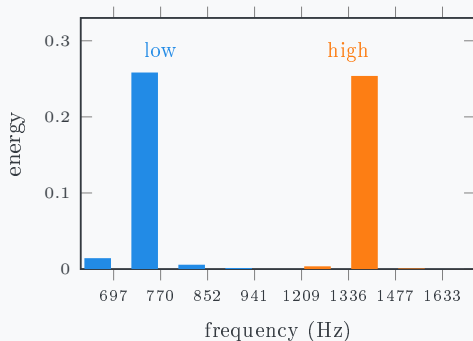
Decoding with Goertzel – from 8 energies to a digit



Energies of one frame of “5”.

1. Loudest row → 770

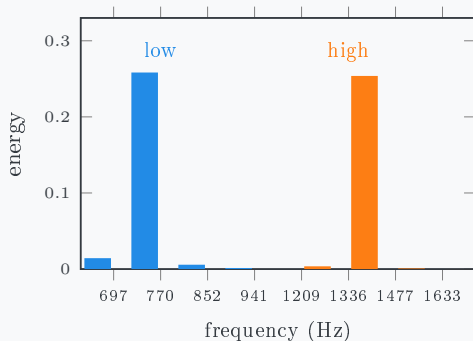
Decoding with Goertzel – from 8 energies to a digit



Energies of one frame of “5”.

1. Loudest row → 770
2. Loudest column → 1336

Decoding with Goertzel – from 8 energies to a digit

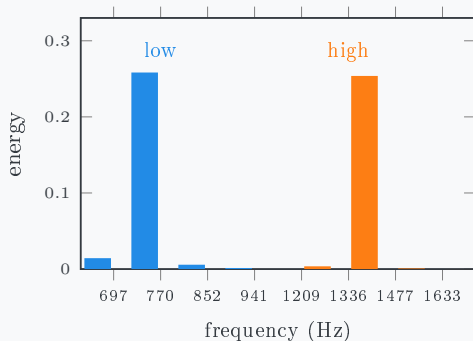


Energies of one frame of “5”.

1. Loudest row → 770
2. Loudest column → 1336
3. Keypad table ⇒

5

Decoding with Goertzel – from 8 energies to a digit



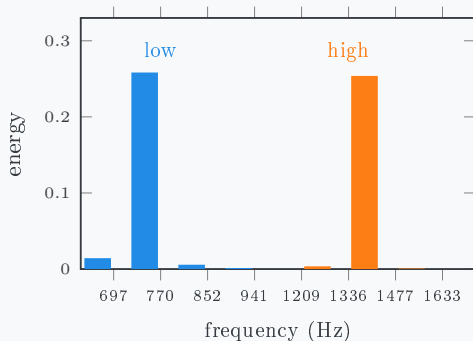
Energies of one frame of “5”.

1. Loudest row → 770
2. Loudest column → 1336
3. Keypad table ⇒

5

4. Is it a real tone? Level -2.9 dB is above floor + 10 dB ✓

Decoding with Goertzel – from 8 energies to a digit



Energies of one frame of “5”.

1. Loudest row $\rightarrow$ 770

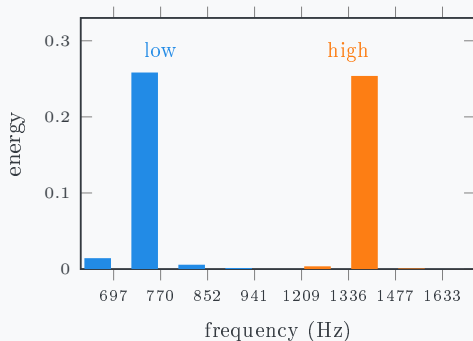
2. Loudest column $\rightarrow$ 1336

3. Keypad table $\Rightarrow$ 5

4. Is it a real tone? Level -2.9 dB is above floor $+10$ dB ✓

5. Dominance $= \frac{0.258+0.254}{0.538} = 0.95 \geq 0.6$ ✓
(a noise-only frame gives 0.38, rejected)

Decoding with Goertzel – from 8 energies to a digit



Energies of one frame of “5”.

1. Loudest row $\rightarrow$ 770
2. Loudest column $\rightarrow$ 1336
3. Keypad table $\Rightarrow$

5

4. Is it a real tone? Level -2.9 dB is above floor $+10$ dB ✓
5. Dominance $= \frac{0.258+0.254}{0.538} = 0.95 \geq 0.6$ ✓
(a noise-only frame gives 0.38, rejected)
6. 6 votes in a row: latch, re-pick on the region (Hann block), validate, emit.

Decoding with FFT – find the peaks, then snap

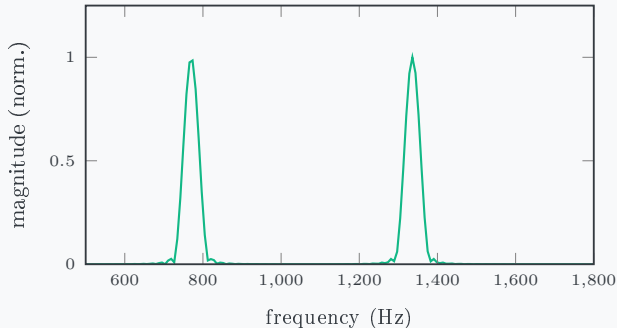
1. Latched region: $45\text{ ms} = 360$ samples, times a Hann window.

Decoding with FFT – find the peaks, then snap

1. Latched region: $45\text{ ms} = 360$ samples, times a Hann window.
2. Zero-pad to 1024: a point every 7.8 Hz.

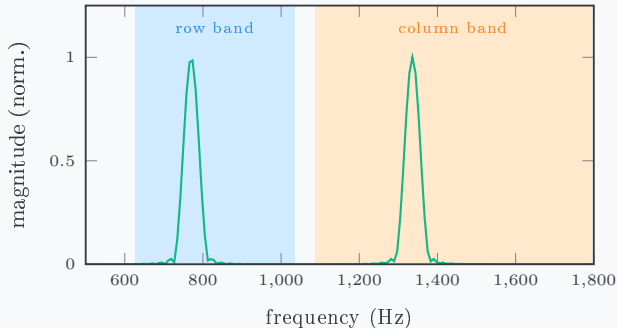
Decoding with FFT – find the peaks, then snap

1. Latched region: 45 ms = 360 samples, times a Hann window.
2. Zero-pad to 1024: a point every 7.8 Hz.
3. `rfft`: energy at every frequency.



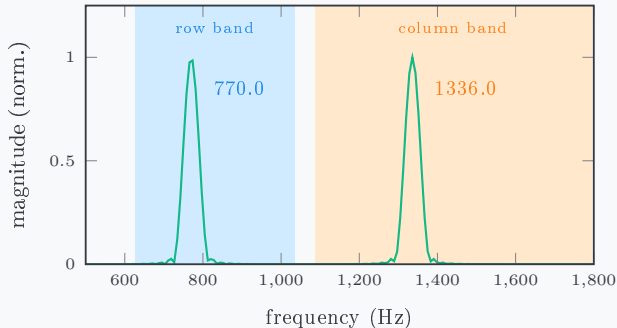
Decoding with FFT – find the peaks, then snap

1. Latched region: 45 ms = 360 samples, times a Hann window.
2. Zero-pad to 1024: a point every 7.8 Hz.
3. `rfft`: energy at every frequency.
4. Tallest peak in the row band and in the column band.



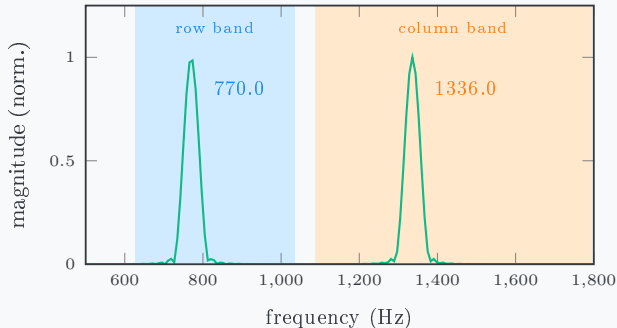
Decoding with FFT – find the peaks, then snap

1. Latched region: 45 ms = 360 samples, times a Hann window.
2. Zero-pad to 1024: a point every 7.8 Hz.
3. `rfft`: energy at every frequency.
4. Tallest peak in the row band and in the column band.
5. Interpolate around each peak:
770.0 Hz, 1336.0 Hz.



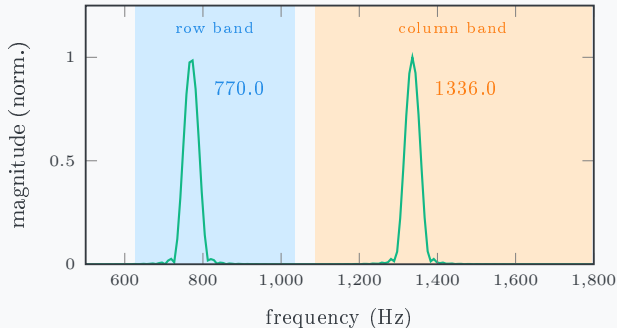
Decoding with FFT – find the peaks, then snap

1. Latched region: 45 ms = 360 samples, times a Hann window.
2. Zero-pad to 1024: a point every 7.8 Hz.
3. `fft`: energy at every frequency.
4. Tallest peak in the row band and in the column band.
5. Interpolate around each peak:
770.0 Hz, 1336.0 Hz.
6. Within 2.5% of a DTMF pair?
Snap to it $\Rightarrow$ 5



Decoding with FFT – find the peaks, then snap

1. Latched region: 45 ms = 360 samples, times a Hann window.
 2. Zero-pad to 1024: a point every 7.8 Hz.
 3. `fft`: energy at every frequency.
 4. Tallest peak in the row band and in the column band.
 5. Interpolate around each peak:
770.0 Hz, 1336.0 Hz.
 6. Within 2.5% of a DTMF pair?
Snap to it $\Rightarrow$ 5
- A sloppy 780 Hz tone still measures 780.0 (1.3% off) and snaps to 770.



Goertzel vs. FFT: same frames, different digit pick

Goertzel

FFT

Goertzel vs. FFT: same frames, different digit pick

Goertzel

- Asks: how much energy at each of the 8 **known** frequencies?

FFT

- Asks: which frequencies are **actually present**?

Goertzel vs. FFT: same frames, different digit pick

Goertzel

- Asks: how much energy at each of the 8 **known** frequencies?
- Output: 8 numbers; argmax of rows and columns

FFT

- Asks: which frequencies are **actually present**?
- Output: whole spectrum, peaks with measured Hz

Goertzel vs. FFT: same frames, different digit pick

Goertzel

- Asks: how much energy at each of the 8 **known** frequencies?
- Output: 8 numbers; argmax of rows and columns
- Cheap, tiny, no padding or power-of-two size

FFT

- Asks: which frequencies are **actually present**?
- Output: whole spectrum, peaks with measured Hz
- Can say “too far from DTMF” and gives spectrum/spectrogram plots

Goertzel vs. FFT: same frames, different digit pick

Goertzel

- Asks: how much energy at each of the 8 **known** frequencies?
- Output: 8 numbers; argmax of rows and columns
- Cheap, tiny, no padding or power-of-two size

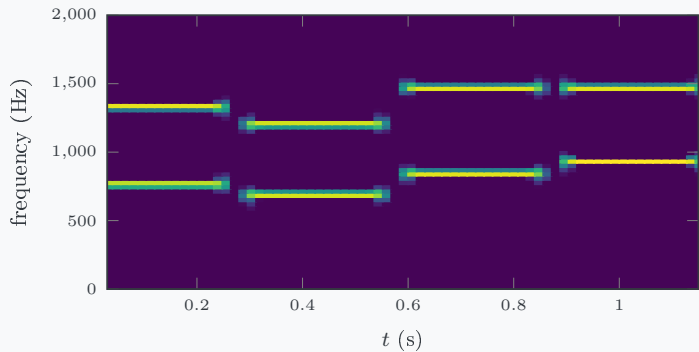
FFT

- Asks: which frequencies are **actually present**?
- Output: whole spectrum, peaks with measured Hz
- Can say “too far from DTMF” and gives spectrum/spectrogram plots

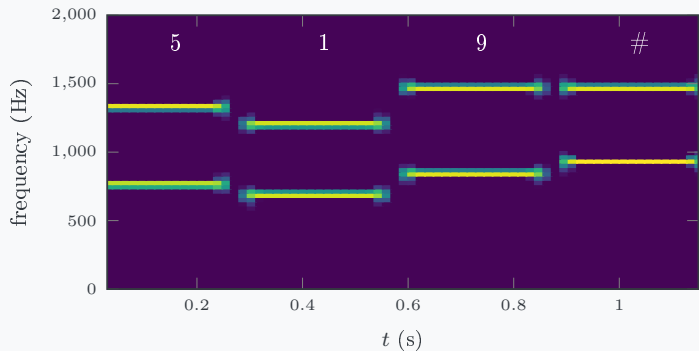
Shared: framing, 8 energies per frame, gate, dominance, 6-vote latch, validation.

Only different: how the digit is picked at latch time.

Phase 7 – Spectrogram: the whole call at a glance

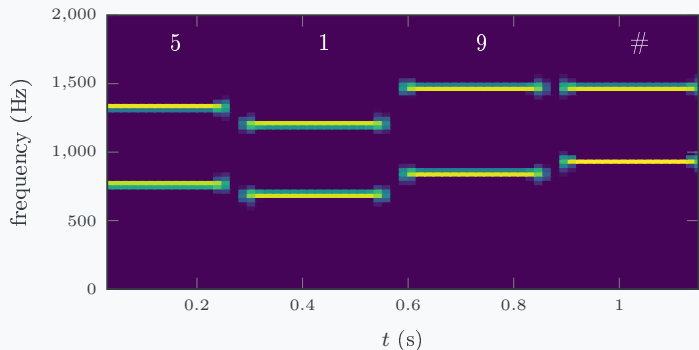


Phase 7 – Spectrogram: the whole call at a glance



Sequence 519#.

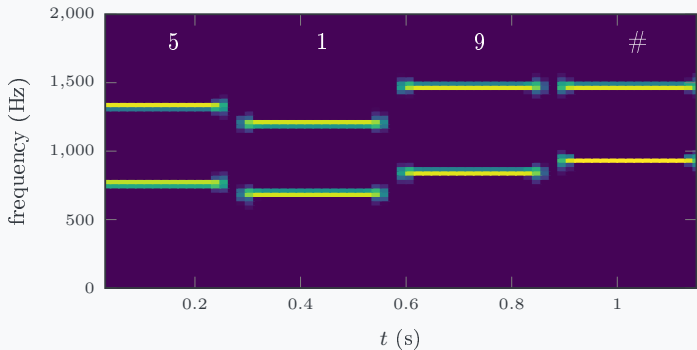
Phase 7 – Spectrogram: the whole call at a glance



Sequence 519#.

Every key = **two horizontal streaks**: one row tone, one column tone.

Phase 7 – Spectrogram: the whole call at a glance



Sequence 519#.

Every key = **two horizontal streaks**: one row tone, one column tone.

e.g. “9” = 852 + 1477 Hz, “#” = 941 + 1477 Hz.

Window-by-window FFT, run in a background thread.

Phase 8/9 – Goertzel is an LTI resonator

Recurrence $\Rightarrow$ characteristic
equation

$$z^2 - 2 \cos \omega z + 1 = 0$$

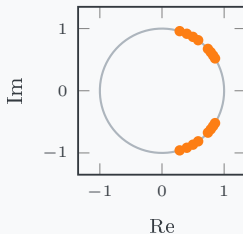
Phase 8/9 – Goertzel is an LTI resonator

Recurrence $\Rightarrow$ characteristic equation

$$z^2 - 2\cos\omega z + 1 = 0$$

Roots $z = e^{\pm j\omega}$: poles **on the unit circle** at $\pm\omega$.

8 frequencies $\rightarrow$ 16 poles



Phase 8/9 – Goertzel is an LTI resonator

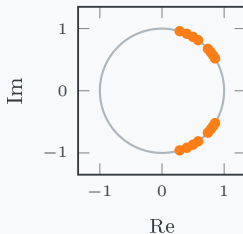
Recurrence $\Rightarrow$ characteristic equation

$$z^2 - 2\cos\omega z + 1 = 0$$

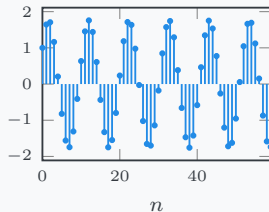
Roots $z = e^{\pm j\omega}$: poles **on the unit circle** at $\pm\omega$.

Hit it with one impulse and it rings at f forever: the impulse response.

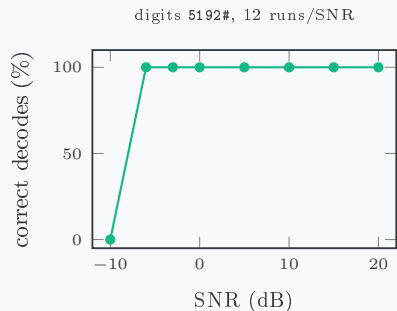
8 frequencies $\rightarrow$ 16 poles



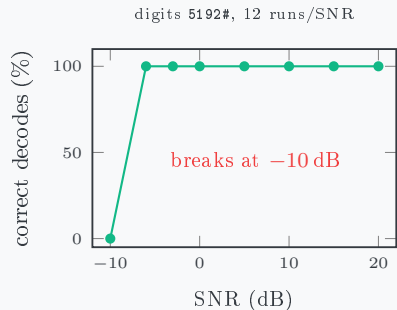
$h[n]$ for 770 Hz



Phase 10 – Breaking it on purpose: noise and aliasing

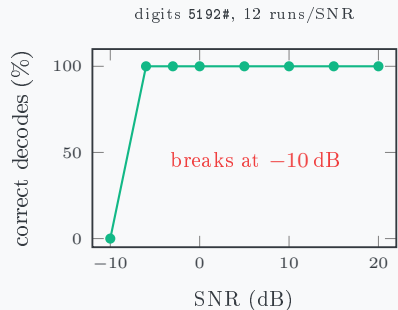


Phase 10 – Breaking it on purpose: noise and aliasing

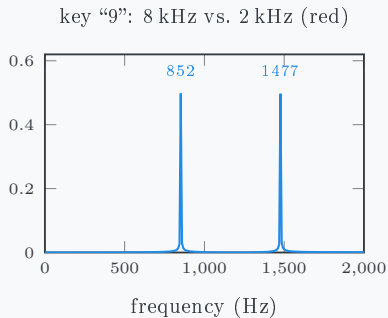


100% from 20 dB down to -6 dB

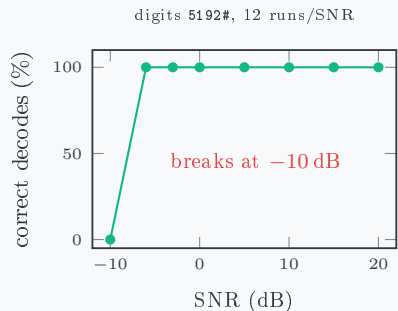
Phase 10 – Breaking it on purpose: noise and aliasing



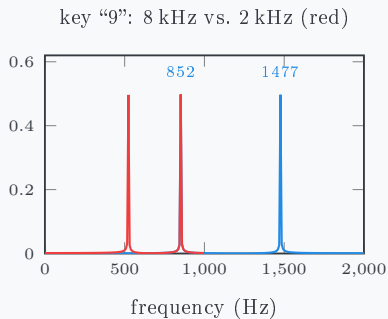
100% from 20 dB down to -6 dB



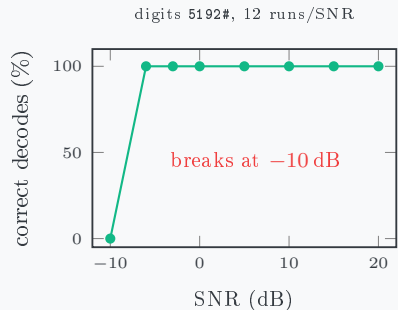
Phase 10 – Breaking it on purpose: noise and aliasing



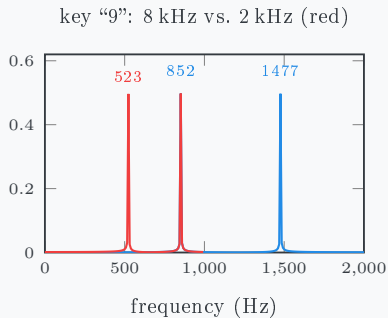
100% from 20 dB down to -6 dB



Phase 10 – Breaking it on purpose: noise and aliasing

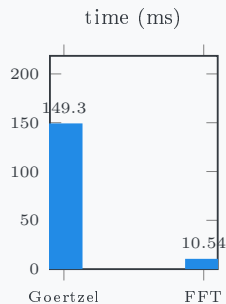


100% from 20 dB down to -6 dB



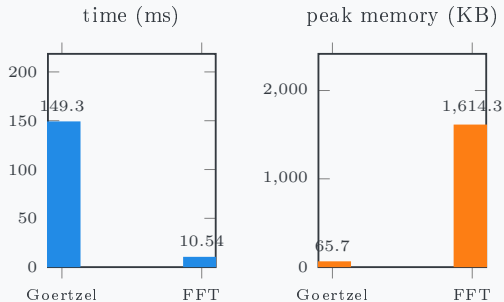
$1477 \text{ Hz} > f_s/2$ folds to $2000 - 1477 = 523 \text{ Hz}$: a wrong tone.

Phase 11 – Benchmark: an honest surprise



Recursive Goertzel is **14×**
slower...

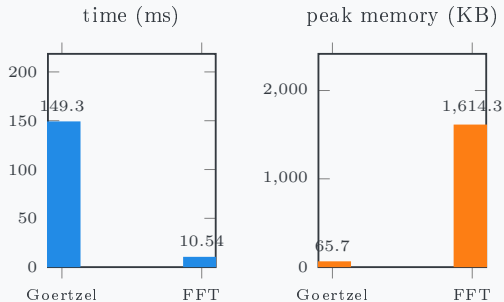
Phase 11 – Benchmark: an honest surprise



Recursive Goertzel is **14×**
slower...

... but uses **25×** less memory.

Phase 11 – Benchmark: an honest surprise

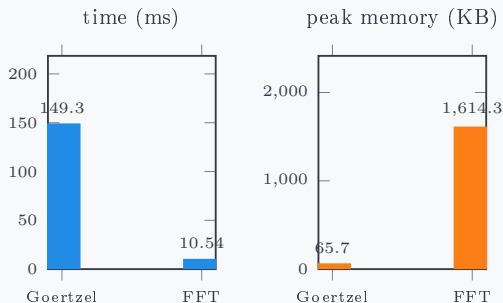


Recursive Goertzel is **14×**
slower...

... but uses **25×** **less memory.**

Why: $s[n]$ needs $s[n-1]$, so a
Python loop, about $0.5\ \mu\text{s}$ per
sample.

Phase 11 – Benchmark: an honest surprise



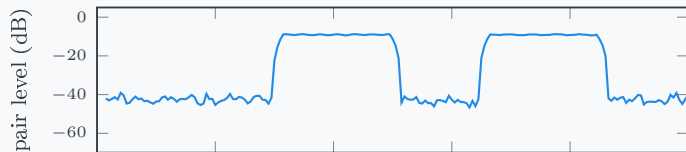
Recursive Goertzel is **14×**
slower...

... but uses **25×** **less memory.**

Why: $s[n]$ needs $s[n-1]$, so a
Python loop, about $0.5 \mu\text{s}$ per
sample.

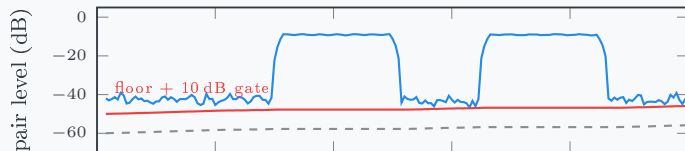
Fix: one bin is a dot product, so
all 8 bins are $\mathbf{W}\mathbf{x}$, a cached
 $8 \times N$ matrix in BLAS.

Engine in action – the tracker on a noisy signal



Real run of DtmfTracker:
noise, "5", gap, "9".

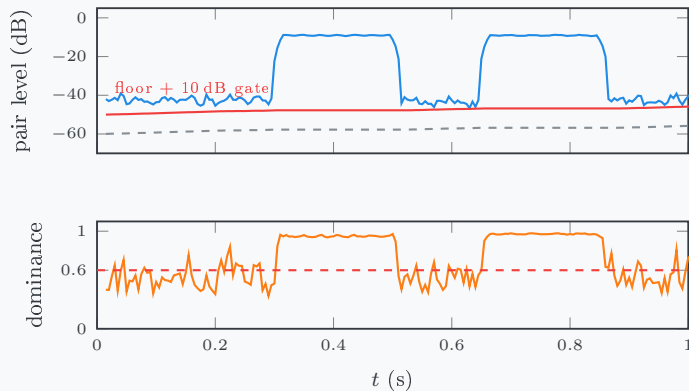
Engine in action – the tracker on a noisy signal



Real run of DtmfTracker:
noise, “5”, gap, “9”.

Gate is relative to a *measured*
noise floor: it adapts to any
room.

Engine in action – the tracker on a noisy signal

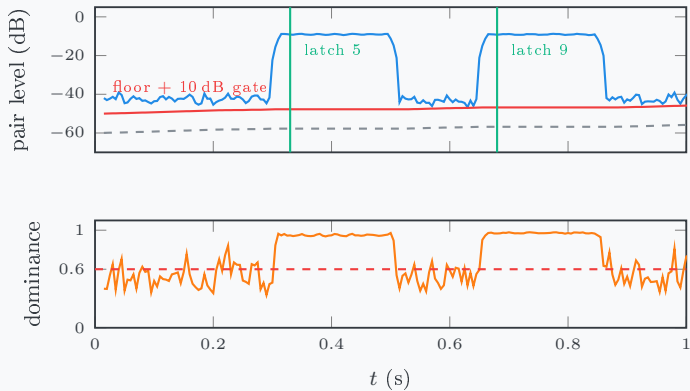


Real run of `DtmfTracker`:
noise, “5”, gap, “9”.

Gate is relative to a *measured*
noise floor: it adapts to any
room.

Tones also need dominance
 ≥ 0.6 .

Engine in action – the tracker on a noisy signal



Real run of `DtmfTracker`:
noise, “5”, gap, “9”.

Gate is relative to a *measured*
noise floor: it adapts to any
room.

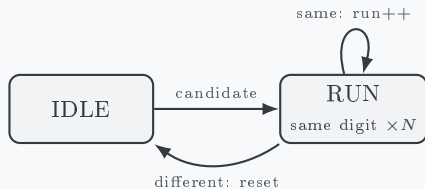
Tones also need dominance
 ≥ 0.6 .

Digit fires **30 ms after tone**
onset, while the key is still
down.

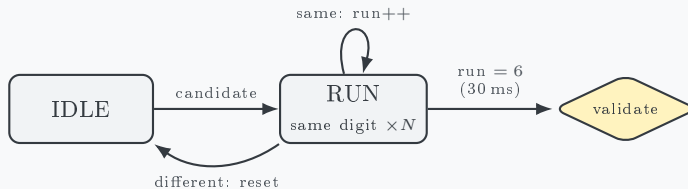
Deciding over time – latch and release



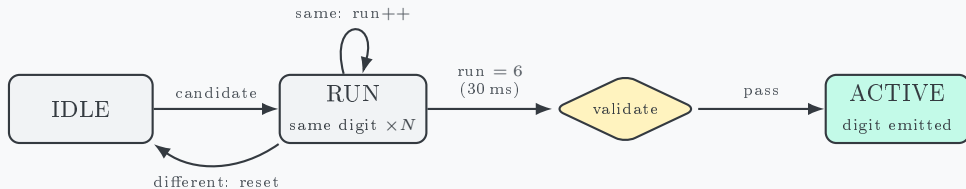
Deciding over time – latch and release



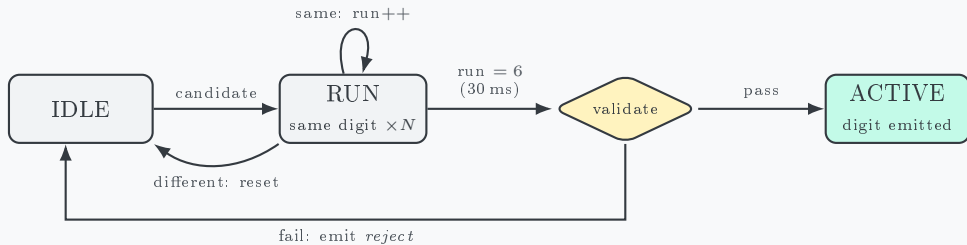
Deciding over time – latch and release



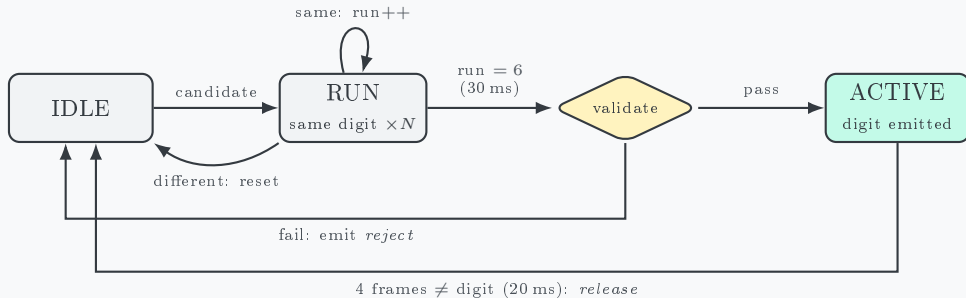
Deciding over time – latch and release



Deciding over time – latch and release



Deciding over time – latch and release



Repeat digits (“55”) need a full latch → release → latch cycle.

Is it really DTMF? (Q.24-style checks)



The diagram shows a signal waveform with two tall, narrow gray vertical bars representing the 350 Hz and 440 Hz components of a dial tone. To the right of these bars are two much shorter bars, one blue and one orange, representing other frequency components. Below the gray bars is the text "350+440".

350+440

1. Dial tone. 350+440 Hz pair as strong as the candidate: reject.

Is it really DTMF? (Q.24-style checks)



1. Dial tone. 350+440 Hz pair as strong as the candidate: reject.



2. Twist. Row vs column within +15/ - 12 dB. Speech and stray beeps fail.

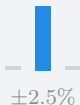
Is it really DTMF? (Q.24-style checks)



1. Dial tone. 350+440 Hz pair as strong as the candidate: reject.

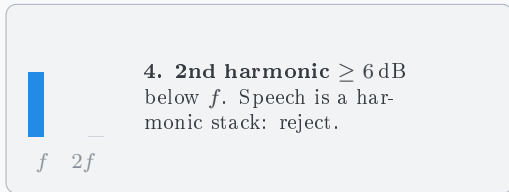
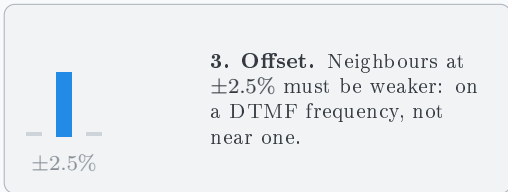
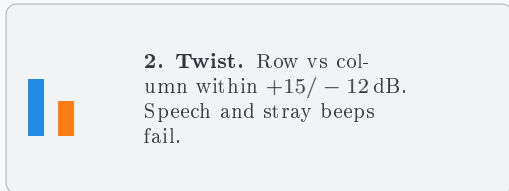


2. Twist. Row vs column within +15/ - 12 dB. Speech and stray beeps fail.



3. Offset. Neighbours at $\pm 2.5\%$ must be weaker: on a DTMF frequency, not near one.

Is it really DTMF? (Q.24-style checks)



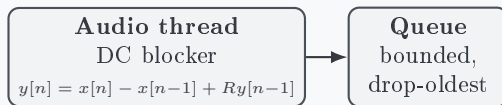
Live decoding – from microphone to screen

Audio thread

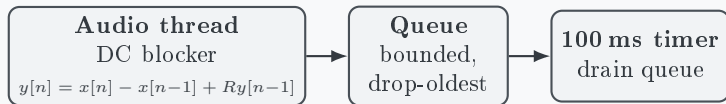
DC blocker

$$y[n] = x[n] - x[n-1] + Ry[n-1]$$

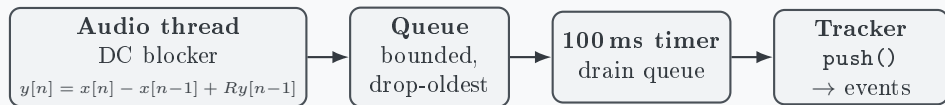
Live decoding – from microphone to screen



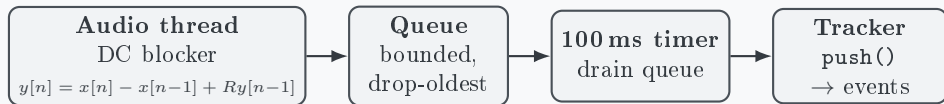
Live decoding – from microphone to screen



Live decoding – from microphone to screen



Live decoding – from microphone to screen



latch **wait for next tick** $\approx 30\text{--}50\text{ ms} + 0\text{--}100\text{ ms} \approx \mathbf{100\text{ ms}}$

the old “segment then wait” design took 350–500 ms

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths
- Synthetic 5192#: 100% correct down to -6 dB SNR

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths
- Synthetic 5192#: 100% correct down to -6 dB SNR
- Live latency budget: digit on screen $\approx$ 100 ms after key press

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths
- Synthetic 5192#: 100% correct down to -6 dB SNR
- Live latency budget: digit on screen $\approx$ 100 ms after key press
- Non-DTMF signals (dial tone, speech-like) are refused by the Q.24-style checks

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths
- Synthetic 5192#: 100% correct down to -6 dB SNR
- Live latency budget: digit on screen $\approx$ 100 ms after key press
- Non-DTMF signals (dial tone, speech-like) are refused by the Q.24-style checks
- Benchmark taught us: implementation (Python loop vs BLAS) can dominate Big-O

What we verified

- All 16 keypad digits decode with **both** the Goertzel and FFT paths
- Synthetic 5192#: 100% correct down to -6 dB SNR
- Live latency budget: digit on screen $\approx$ 100 ms after key press
- Non-DTMF signals (dial tone, speech-like) are refused by the Q.24-style checks
- Benchmark taught us: implementation (Python loop vs BLAS) can dominate Big-O

Demo: keypad $\rightarrow$ waveform & spectrum $\rightarrow$.wav $\rightarrow$ live mic

Thank you. Questions?

Backup – Our FFT: Cooley–Tukey butterflies ($N = 4$)

bit-reversed input

$x[0]$ •

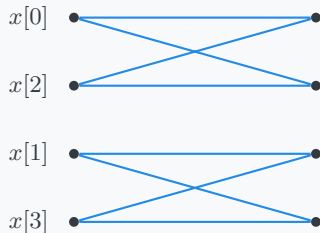
$x[2]$ •

$x[1]$ •

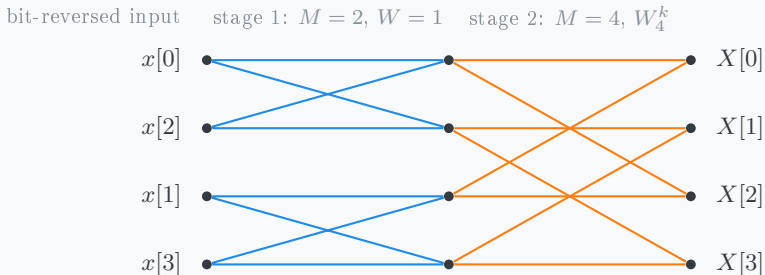
$x[3]$ •

Backup – Our FFT: Cooley–Tukey butterflies ($N = 4$)

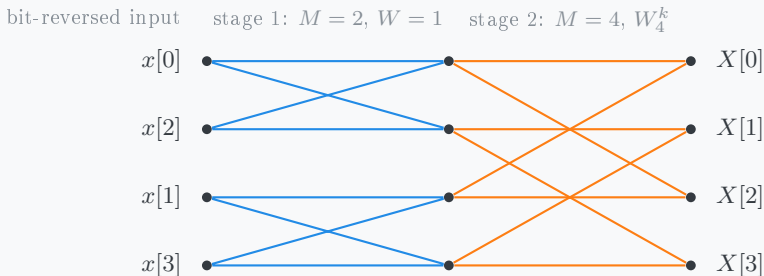
bit-reversed input stage 1: $M = 2, W = 1$



Backup – Our FFT: Cooley–Tukey butterflies ($N = 4$)

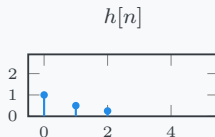
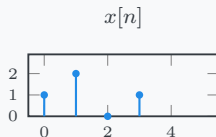


Backup – Our FFT: Cooley–Tukey butterflies ($N = 4$)

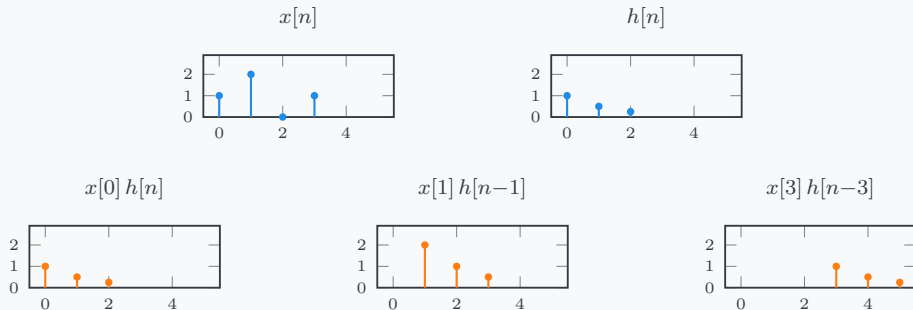


$\log_2 N$ stages of $N/2$ butterflies $\Rightarrow N \log_2 N$ instead of N^2 . The permutation is
`bit_reversal` in `utils.py`.

Backup – Convolution by superposition (signal_lti.py)

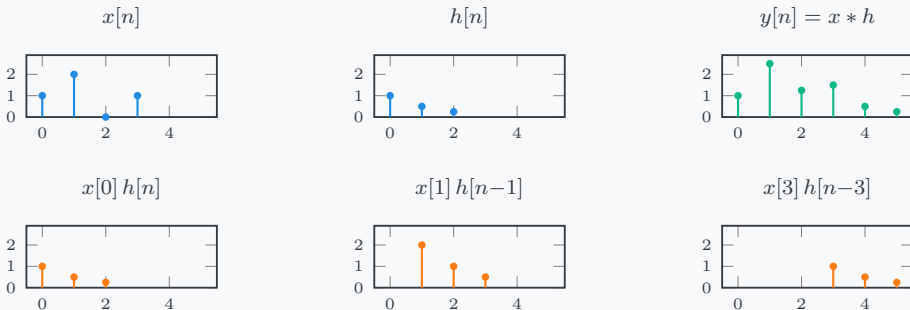


Backup – Convolution by superposition (signal_lti.py)



Each nonzero input sample launches a scaled, shifted copy of h ;

Backup – Convolution by superposition (signal_lti.py)



Each nonzero input sample launches a scaled, shifted copy of h ; the sum of the copies is $y[n]$.

Backup – FFT decoder: sub-bin accuracy

Hann window (less leakage)

Backup – FFT decoder: sub-bin accuracy

Hann window (less leakage)

→ zero-pad $\times 2$ → `rfft`

Backup – FFT decoder: sub-bin accuracy

Hann window (less leakage)

→ zero-pad $\times 2$ → `rfft`

→ tallest peak per band

Backup – FFT decoder: sub-bin accuracy

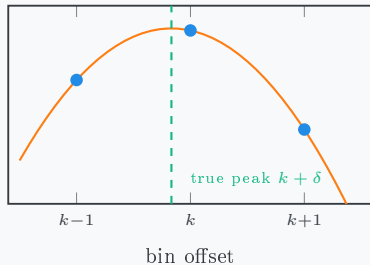
Hann window (less leakage)

→ zero-pad $\times 2$ → `rfft`

→ tallest peak per band

→ **parabolic interpolation** of 3 bins

$$\delta = \frac{1}{2} \frac{a-c}{a-2b+c}, \text{ peak at } k + \delta$$



Backup – FFT decoder: sub-bin accuracy

Hann window (less leakage)

→ zero-pad $\times 2 \rightarrow \text{rfft}$

→ tallest peak per band

→ **parabolic interpolation** of 3 bins

→ nearest DTMF pair within 2.5%

$\delta = \frac{1}{2} \frac{a-c}{a-2b+c}$, peak at $k + \delta$

